Practical No. 2

Mapped CO: CO1 Unit I Bloom Level: L3 – Apply

1.0) AIM

Write a program to create a chain of at least 3 blocks containing one Genesis block, where all other blocks are connected by the hash function.

2.0) SCOPE

To understand how a chain of blocks is created and how each block is linked to the previous one using its hash.

3.0) FACILITIES / APPARATUS

- **Computer System:** Any desktop or laptop computer with basic configuration.
- **Software Package:** Python 3 (IDLE), with the built-in hashlib library (no installation required).

4.0) THEORY

A blockchain is a growing list of records, called blocks, that are linked to one another. Bitcoin was the first successful application of this system, and many other cryptocurrencies were later built on the same idea. However, the system is not limited to financial data — any type of data can be stored in a blockchain.

Data stored in a blockchain should have the following characteristics:

- Immutable — once written, a block cannot be changed.
- Distributed — copies of the chain are shared across many nodes.
- Persistent — there is no loss of data.
- Tamper-evident — any change is easy to detect.

How the blocks are linked:

Each block stores three important things: its data, the hash of the previous block, and its own hash (calculated from its contents). The very first block is called the Genesis block; it has no previous block, so its previous-hash is set to a fixed value such as “0”. Every block after it stores the hash of the block before it. This is what forms the “chain”: if any earlier block is changed, its hash changes, which breaks the link to all the blocks that follow — making the blockchain tamper-evident.

Steps to build the blockchain:

1. **Create a Block class** that stores the index, data, previous hash, and calculates its own hash.
2. **Write a hash function** that uses SHA-256 to hash the block’s contents.
3. **Create the Genesis block** with a fixed previous-hash of “0”.
4. **Add more blocks,** each one using the previous block’s hash to stay linked.
5. **Print the chain** to see that each block’s “Previous Hash” matches the previous block’s “Block Hash”.

5.0) PROGRAM

```
import hashlib
```

```
# A block stores its data, the previous block's hash, and its own hash
class Block:
    def __init__(self, index, data, previous_hash):
        self.index = index
        self.data = data
        self.previous_hash = previous_hash
        self.hash = self.calculate_hash()

    def calculate_hash(self):
        text = str(self.index) + str(self.data) + str(self.previous_hash)
        return hashlib.sha256(text.encode()).hexdigest()


# Create the blockchain as a list of blocks
blockchain = []

# Block 0 = Genesis block (no previous block, so previous_hash = "0")
genesis = Block(0, "Genesis Block", "0")
blockchain.append(genesis)

# Each new block uses the previous block's hash to stay linked
block1 = Block(1, "Satoshi -> Alex : 10 BTC", blockchain[-1].hash)
blockchain.append(block1)

block2 = Block(2, "Alex -> Lucy : 2 BTC", blockchain[-1].hash)
blockchain.append(block2)

# Print the chain so the linkage is visible
for block in blockchain:
    print("Index         :", block.index)
    print("Data          :", block.data)
    print("Previous Hash :", block.previous_hash)
    print("Block Hash    :", block.hash)
    print("-" * 70)
```

6.0) EXPECTED OUTPUT

The program prints the three blocks. Notice that each block's "Previous Hash" is exactly equal to the previous block's "Block Hash" — this is the link that forms the chain. (Hash values will look like the ones below.)

```
Index         : 0
Data          : Genesis Block
Previous Hash : 0
Block Hash    : d894de287307c6e646599a69ef054fe0bf9c157e46d2d396232d320dbe326b3d
----------------------------------------------------------------------
Index         : 1
Data          : Satoshi -> Alex : 10 BTC
Previous Hash : d894de287307c6e646599a69ef054fe0bf9c157e46d2d396232d320dbe326b3d
Block Hash    : 770516d0f42ef4e627bf64109eabdfe444ec46b752261340bd295a0fa0d49e05
----------------------------------------------------------------------
Index         : 2
Data          : Alex -> Lucy : 2 BTC
Previous Hash : 770516d0f42ef4e627bf64109eabdfe444ec46b752261340bd295a0fa0d49e05
Block Hash    : 14c5acb8d3304024efe381afdbf5f694bd2fd9695a031b33dffa2aadde338879
----------------------------------------------------------------------
```

Note: *The Genesis block's hash stays the same every run, but the link is what matters — Block 1's Previous Hash = Block 0's Block Hash, and Block 2's Previous Hash = Block 1's Block Hash.*

7.0) CONCLUSION

In this way, we created a chain of at least three blocks containing one Genesis block, where all other blocks are connected by the hash function — each block storing the hash of the block before it to form a tamper-evident chain.